GPS

Daniel Zuñiga
Rubén Parra
Iván Escandón
Pablo Soriano
Luis Fernando

1. Introducción **3**
2. Objetivos del proyecto **3**
3. Descripción del sistema **4**
4. Representación del grafo **4**
4.1 Nodos 4
4.2 Aristas 4
4.3 Pesos o costes 4
5.1 Archivo CSV 5
5.2 Base de datos SQLite 5
6. Patrón DAO (capa de acceso a datos) **5**
7.1 Cómo funciona 6
7.2 Reconstrucción de la ruta 6
8. Ruta alternativa (15%) **6**
9. Destinos intermedios **7**
10. Interacción con el usuario **7**
11. Histórico de rutas **8**
12. Casos de prueba **8**

1. Introducción

En este proyecto final se va a desarrollar una aplicación en **Python** que simula un **GPS**, es decir, un sistema capaz de calcular la mejor ruta entre dos puntos.

La idea principal es trabajar con **grafos ponderados dirigidos** y aplicar el algoritmo de **Dijkstra**, que sirve para encontrar el camino más corto entre dos nodos.

El mapa representará una red de carreteras realista, con intersecciones y carreteras que pueden ser de un solo sentido o de doble sentido.

2. Objetivos del proyecto

Los objetivos principales de este proyecto son:

- Crear un mapa representado como un **grafo dirigido con pesos**.
- Tener al menos **40 nodos** y mínimo **80 aristas**.
- Asegurar que al menos **40 aristas sean unidireccionales** (solo un sentido).
- Implementar el algoritmo de **Dijkstra** para calcular la ruta óptima.
- Leer los datos desde un archivo **CSV**, pero no usarlo directamente para calcular rutas.
- Guardar toda la información del grafo en una base de datos **SQLite**.
- Usar el patrón **DAO** para separar la parte de base de datos del resto del programa.
- Guardar un historial de rutas calculadas para no repetir cálculos si ya se hizo antes.
- Permitir al usuario elegir una ruta alternativa si es parecida a la mejor.
- Permitir añadir un destino intermedio.

3. Descripción del sistema

El programa funcionará como un GPS sencillo. Una vez cargado el mapa, el usuario elegirá:

- un punto de origen
- un punto de destino

Con esos datos, el programa calculará el camino más corto usando Dijkstra y mostrará:

- el coste total del recorrido
- el camino completo (lista de nodos por donde pasa)

Si no existe ruta posible entre origen y destino, el programa lo dirá claramente.

4. Representación del grafo

El mapa se representa con un **grafo dirigido ponderado**, donde se diferencian nodos, aristas y costes.

4.1 Nodos

Los nodos representan ciudades, cruces o puntos importantes del mapa.
Cada nodo tendrá un identificador único para poder referenciarlo.

4.2 Aristas

Las aristas representan las carreteras que unen un nodo con otro.
Como el grafo es dirigido, significa que puede haber carreteras de:

- un solo sentido (A → B)
- doble sentido (A → B y B → A)

4.3 Pesos o costes

Cada carretera tiene un coste asociado, que puede representar:

- distancia (km)
- tiempo (minutos)

5. Carga de datos (CSV y SQLite)

El proyecto empieza con un fichero CSV, pero la información se guardará en una base de datos.

5.1 Archivo CSV

El CSV tendrá los datos de:

- nodos
- conexiones (aristas)
- coste asociado

Este archivo solo sirve para cargar los datos al principio.

5.2 Base de datos SQLite

El programa creará y usará una base de datos SQLite para guardar toda la información del grafo.
Luego, el cálculo de rutas se hará sacando la información desde SQLite mediante consultas SQL

6. Patrón DAO (capa de acceso a datos)

Para hacerlo bien estructurado, se usará el patrón **DAO**, que básicamente sirve para que el programa esté ordenado.

Esto significa que:

- Dijkstra no tendrá SQL dentro
- Dijkstra no abrirá conexiones a la base de datos
- Todo lo relacionado con consultas y base de datos estará en una clase o funciones aparte

El DAO se encargará de:

- conectarse a SQLite
- ejecutar consultas
- devolver los datos al programa

7. Algoritmo de Dijkstra

El algoritmo principal del proyecto es **Dijkstra**, que sirve para calcular la ruta más corta en un grafo con pesos positivos.

7.1 Cómo funciona

Dijkstra funciona calculando el coste mínimo desde el nodo origen a todos los demás, y va actualizando las distancias según encuentra caminos mejores.

Para hacerlo eficiente se usará:

- una cola de prioridad
- un diccionario de distancias
- un diccionario de predecesores

7.2 Reconstrucción de la ruta

Cuando se llega al destino, se reconstruye el camino usando el diccionario de predecesores.

Así se puede obtener la ruta completa desde el origen hasta el destino, mostrando todos los nodos.

8. Ruta alternativa (15%)

Si existe otra ruta que sea casi igual de buena, el programa la podrá ofrecer.

La condición es que el coste de esa ruta alternativa no sea superior al **15%** de diferencia con respecto a la ruta óptima.

Ejemplo:

- ruta óptima: 10 km
  alternativa válida: hasta 11-12 km
- ruta óptima: 30 min
  alternativa válida: hasta 33-36 min

Si existe una alternativa, el usuario podrá elegir cuál quiere.

9. Destinos intermedios

El sistema permitirá al usuario añadir al menos un destino intermedio, por ejemplo:

Origen → Intermedio → Destino

En este caso el programa calculará dos rutas:

- origen a intermedio
- intermedio a destino

Y luego unirá ambas para mostrar la ruta completa.

10. Interacción con el usuario

La parte de interacción con el usuario será la encargada de:

- pedir el origen y el destino
- pedir si se quiere un punto intermedio
- mostrar la ruta final y el coste
- mostrar errores si no existe camino
- preguntar si se quiere usar la ruta alternativa

Esta parte no implementa Dijkstra ni la estructura del grafo, solo llama a las funciones necesarias.

11. Histórico de rutas

El programa guardará las rutas consultadas en una tabla de la base de datos.

Cada registro guardará:

- origen
- destino
- fecha y hora de consulta
- coste total
- si el usuario eligió alternativa o no

Esto sirve para que si el usuario vuelve a pedir el mismo trayecto, el programa pueda usar el historial en vez de volver a ejecutar Dijkstra.

12. Casos de prueba

Se incluirán diferentes pruebas para demostrar que funciona correctamente:

- ruta normal entre dos nodos
- ruta con alternativa dentro del 15%
- ruta con destino intermedio
- ruta donde no exista conexión posible
- ruta repetida para comprobar que se usa el historial